

```

1  /* =====
2      datenbereinigung.js – Datenbereinigung (reines JS)
3      CAS Generative Data Design: Datenaufbereitung (Python, siehe data-prep/)
4      → Datenbereinigung (hier, reines JS) → Zeichnen (sketch.js, p5).
5      Enthält ausschliesslich reine Datenfunktionen – keine p5-Zeichenaufrufe.
6  ===== */
7
8  // --- Modulkapselung -----
9  // 14 von 52 Namen intern, 38 exportiert. Konvention: docs/architektur.md.
10 // ACHTUNG Skript 1 in index.html. kreisgrafik.js liest hexZuRgb beim Laden
11 // – diese Datei nach hinten schieben bricht kreisgrafik.js.
12 (function () {
13
14 // Wortlaut wie in docs/topografie-der-gefuehle-grafik.pdf. Gelesen von der
15 // Legende (kreisgrafik.js) und der Annotationsleiste (sketch.js) – beide
16 // müssen dasselbe Wort zeigen.
17 const CATEGORY_LABELS = {
18     gold_dunkel: 'Raum und Umwelt',
19     gold_mittel: 'Stimmung und Emotion',
20     gold_hell: 'Gesellschaft und Soziales',
21 };
22 const ROUTE_COLOR = '#63561F';
23
24 // Hex-Farbstring zu r/g/b. Nötig, weil p5s stroke() Hex und Alpha nicht
25 // gemeinsam annimmt, die Route aber variables Alpha braucht.
26 function hexZuRgb(hex) {
27     let bereinigt = hex.replace('#', '');
28     return {
29         r: parseInt(bereinigt.substring(0, 2), 16),
30         g: parseInt(bereinigt.substring(2, 4), 16),
31         b: parseInt(bereinigt.substring(4, 6), 16),
32     };
33 }
34 const ROUTE_COLOR_RGB = hexZuRgb(ROUTE_COLOR);
35
36 // Gegenrichtung: r/g/b-Tripel zu Hexstring. Nötig, wo eine Farbe als String
37 // gebraucht wird – der CSS-Verlauf der Annotationsleiste, der strokeStyle des
38 // Icons, drawHatchedCircle.
39 function rgbZuHex(rgb) {
40     return '#' + rgb.map(v => Math.round(v).toString(16).padStart(2, '0')).toUpperCase().join('');
41 }
42
43 // Ein einziges Orange für alles, was F-Wert heisst: die Punkte an den Kreisen,
44 // ihre Beschriftungen in der Legende, die Kapitelpunkte und die Routen-Hitze im
45 // Übersichtsakt, der Balken der Annotationsbox. Früher standen hier drei
46 // Abstufungen je F-Wert-Typ und ein eigener, dunklerer Ton für die Punkte –
47 // dadurch zeigte die Legende eine andere Farbe als die Karte daneben.
48 const FWERT_COLOR = '#C2511C';
49 const FWERT_COLOR_RGB = hexZuRgb(FWERT_COLOR);
50
51 // Punktgrösse 1..3 je F-Wert-Typ. Der seltene vierte Typ
52 // (persoenliche_sehnsucht, 1 Annotation) fehlt und fällt auf 1 zurück.
53 const FWERT_PUNKTGROESSE = {
54     ort_loest_emotion_aus: 1, // Raum löst Emotion aus
55     emotion_faerbt_raum: 2, // Emotion färbt Raum
56     koerper_als_sensor: 3, // Körper als Sensor
57 };
58
59 // Pixel-Durchmesser je Punktgrösse. Steht hier neben FWERT_PUNKTGROESSE, weil
60 // beide dasselbe Mass in zwei Schritten ausdrücken; gelesen von kreisgrafik.js
61 // (die Punkte selbst) und fotomarker.js (leitet daraus seine Markergrösse ab).
62 const FWERT_PUNKT_DURCHMESSER = { 1: 5, 2: 7.5, 3: 10 };
63
64 // Ausformulierte Namen der drei F-Wert-Typen, Gegenstück zu CATEGORY_LABELS.
65 // Wortlaut wie im PDF, ohne das frühere Präfix «Wechselwirkung:».
66 const FWERT_LABELS = {

```

```

67   ort_loest_emotion_aus: 'Der Raum löst die Emotion aus',
68   emotion_faerbt_raum: 'Die Emotion beeinflusst die Raumwahrnehmung',
69   koerper_als_sensor: 'Der Körper spürt',
70 };
71
72 // Die drei Valenzgruppen der F-Wert-Punkte, benannt wie im PDF.
73 // ACHTUNG das PDF schreibt «Neutral Wahrnehmung» – Tippfehler, hier bewusst
74 // korrigiert. Nicht «zurückkorrigieren», wenn jemand gegen das PDF abgleicht.
75 const WAHRNEHMUNG_LABELS = {
76   pos: 'Positive Wahrnehmung',
77   neg: 'Negative Wahrnehmung',
78   neutral: 'Neutrale Wahrnehmung',
79 };
80
81 // Überschriften der beiden Legendenblöcke. Im PDF steht «GEFÜHLSKATEGORIEN»
82 // nur in der Textebene und ist weiss gerendert, also unsichtbar; hier wird sie
83 // sichtbar gezeichnet, spiegelbildlich zu «KÖRPER UND RAUM».
84 const LEGENDE_BLOCK_TITEL = {
85   kategorien: 'GEFÜHLSKATEGORIEN',
86   fwerte: 'KÖRPER UND RAUM',
87 };
88
89 // Beschriftungen rund um den Legendenkreis, ebenfalls wortgetreu aus dem PDF.
90 // Die Kreisgrösse steht mehrzeilig, damit der Block rechts nicht überbreit wird.
91 const LEGENDE_KREISGROESSE = [
92   'Kreisgrösse:',
93   'Anzahl Gefühlsäusserungen',
94   'an diesem Ort',
95   'Der Kreis wächst mit jedem',
96   'geäusserten Gefühl.',
97 ];
98 const LEGENDE_VALENZ = { pos: 'Anteil positiver Gefühle', neg: 'Anteil negativer Gefühle' };
99 const LEGENDE_ORTSBESCHRIFTUNG = 'ORTSBESCHRIFTUNG';
100
101 // Kopf der Legende, im PDF auf jeder Seite gleich.
102 const LEGENDE_TITEL = 'TOPOGRAFIE DER GEFÜHLE';
103 const LEGENDE_UNTERTITEL = 'Legende';
104
105 // Fotomarker: dunkles Blaugrau, bewusst NICHT aus der Orange-Reihe. Sonst
106 // stünden auf derselben Karte drei runde orange Zeichen mit drei Bedeutungen
107 // – F-Wert-Punkte, Kapitelpunkte und Fotomarker.
108 const FOTO_MARKER_FARBE = '#3A5058';
109 const FOTO_MARKER_FARBE_RGB = hexZuRgb(FOTO_MARKER_FARBE);
110 // Heller Kern im Marker, damit er auch auf dunklem Untergrund als Ring liest.
111 // Derselbe Ton wie der Grund der Graph-Ansicht, aber eine eigene Entscheidung.
112 const FOTO_MARKER_KERN_FARBE = '#E2E6E1';
113 const FOTO_MARKER_KERN_FARBE_RGB = hexZuRgb(FOTO_MARKER_KERN_FARBE);
114
115 // Schriften fürs Canvas. Spiegeln --sans/--serif in style.css; p5 kennt die
116 // CSS-Variablen nicht, deshalb hier als Literal.
117 const SCHRIFT_SANS = "'Source Sans 3', sans-serif";
118 const SCHRIFT_SERIF = "'Source Serif 4', serif";
119
120 // Harmonische Reihe: gleiche Sättigung und Helligkeit, Hue wandert 44°–50°.
121 // Die Schlüssel bleiben, daran hängt zaehleBandCounts.
122 const KREIS_KATEGORIEN = [
123   { key: 'gold_dunkel', farbe: [198, 162, 43] },
124   { key: 'gold_mittel', farbe: [188, 148, 143] },
125   { key: 'gold_hell', farbe: [52, 64, 92] },
126 ];
127
128 // Dieselben drei Farben als Hexstrings, für die Aufrufer, die keine Tripel
129 // nehmen (Annotationsleiste in sketch.js, Icon in kreisgrafik.js).
130 // ACHTUNG abgeleitet, nicht zweitgeschrieben: als eigene Liste driften die
131 // beiden Schreibweisen auseinander, sobald jemand nur eine davon anfasst.
132 const CATEGORY_COLORS = Object.fromEntries(

```

```

133     KREIS_KATEGORIEN.map(kat => [kat.key, rgbZuHex(kat.farbe)]));
134
135     // Hat Kapitel X ein Spine-Panel? Kapitel 01 fehlt, es hat sein eigenes.
136     // Welche Orte darin stehen, entscheidet ortRunsFuerSpine() weiter unten.
137     const KAPITEL_MIT_SPINE_PANEL = new Set([
138         '02', '03', '04', '05', '06', '07', '08', '09', '10',
139         '11', '12', '13', '14', '15', '16', '17', '18',
140     ]);
141
142     // "Wohnung Duroy" und vier Mini-Erwähnungen daneben werden zu einem Punkt
143     // zusammengefasst; "Rue Notre-Dame de Lorette" bleibt ein eigener.
144     // ACHTUNG die Zuordnung läuft über die Erzählposition (ai), nicht über den
145     // ortBasis-Text: vor Annotation id 8 Sammelpunkt, ab id 8 die Strasse.
146     const WOHNUNG_SAMMELPUNKT_ANKER = 'Lokal in der Nähe der Rue Notre-Dame de Lorette';
147     const RUE_NOTRE_DAME_DE LORETTE_ORT = 'Rue Notre-Dame de Lorette';
148     const WOHNUNG_SPLIT_ANNOTATION_ID = 8; // "Daraufhin ging er die Rue Notre-Dame de Lorette
149     hinunter."
150     const WOHNUNG_SAMMELPUNKT_ABSORBIERTE_ORTRUNS = new Set([
151         'Lokal mit festen Preisen nahe Rue Notre-Dame de Lorette',
152         'Straße nahe Rue Notre-Dame de Lorette',
153         'Boulevard',
154         'Rue Notre-Dame de Lorette / Paris',
155     ]);
156
157     // Erzählposition der Split-Annotation, siehe ACHTUNG oben.
158     function wohnungSplitAi(daten = stationenData) {
159         let ai = daten.annotationen.findIndex(a => a.id === WOHNUNG_SPLIT_ANNOTATION_ID);
160         return ai === -1 ? Infinity : ai;
161     }
162
163     const WOHNUNG_VOR_SPLIT_FILTER = (a, ai) => a.station === 0 && ai < wohnungSplitAi();
164     const RUE_NOTRE_DAME_FILTER = (a, ai) => a.station === 0 && ai >= wohnungSplitAi();
165
166     // Liefert je Ort den passenden Filter: Positions-Filter beim Wohnung-Split,
167     // Set aus Ort plus zugehörigen Gedanken-Orten, sonst der ortBasis-String.
168     function wohnungFilterFuerOrt(ort) {
169         if (ort === WOHNUNG_SAMMELPUNKT_ANKER) return WOHNUNG_VOR_SPLIT_FILTER;
170         if (ort === RUE_NOTRE_DAME_DE LORETTE_ORT) return RUE_NOTRE_DAME_FILTER;
171         let gedankenQuellen = Object.keys(GEDANKEN_ZIEL_ORT).filter(quelle => GEDANKEN_ZIEL_ORT[quelle]
172         === ort);
173         if (gedankenQuellen.length > 0) return new Set([ort, ...gedankenQuellen]);
174         return ort;
175     }
176
177     // Die fünf gedachten Orte in Kapitel 1. Nur die Werte werden gelesen, die
178     // Schlüssel nennen den ortBasis-Text im Datensatz.
179     const GEDANKEN_FILTER = {
180         'Champs-Élysées / Avenue du Bois de Boulogne': 'Champs-Élysées / Avenue du Bois de Boulogne',
181         'Afrika (Erinnerung, Militärdienst)': 'Afrika',
182         'Bois de Boulogne, Paris': 'Bois de Boulogne',
183         'Parc Monceau, Paris': 'Parc Monceau',
184         'imaginiertes Sommergarden, Paris': 'imaginiertes Sommergarden',
185     };
186
187     // Diese fünf bekommen keinen eigenen Kreis; sie zählen beim echten Ort mit,
188     // an dem Duroy gerade steht (GEDANKEN_ZIEL_ORT).
189     const GEDANKEN_ORTRUN_UNTERDRUECKT = new Set(
190         Object.values(GEDANKEN_FILTER).filter(v => typeof v === 'string')
191     );
192
193     // Wohin eine gedachte Annotation zählt: zum letzten echten Ort davor in
194     // derselben station-Gruppe, wo Duroy physisch steht.
195     const GEDANKEN_ZIEL_ORT = {
196         'Champs-Élysées / Avenue du Bois de Boulogne': 'Boulevard Poissonnière',
197         'Afrika': 'Place de la Madeleine',
198         'Bois de Boulogne': 'Café Américain',

```

```
197   'Parc Monceau': 'Café Américain',
198   'imaginierter Sommergarten': 'Café Américain',
199 };
200
201 // Unterdrückt dieser Ort seinen eigenen Auftritt? Betrifft die absorbierten
202 // Wohnung-Erwähnungen und die Gedanken-Orte.
203
204 // ACHTUNG das Gate daten === stationenData muss bleiben: beide Sets sind
205 // reine Namenslisten ohne Kapitelbezug, sonst verschluckt z.B. Kapitel 3
206 // seinen echten "Parc Monceau".
207 function istKapitel1Unterdrueckt(ort, daten) {
208   if (daten !== stationenData) return false;
209   return WOHNUNG_SAMMELPUNKT_ABSORBIERTE_ORTRUNS.has(ort)
210     || GEDANKEN_ORTRUN_UNTERDRUECKT.has(ort);
211 }
212
213 // ortRun-Namen mit eigenem Spine-Eintrag: jeder Kartenkreis, ohne die oben
214 // unterdrückten.
215 function ortRunsFuerSpine(daten) {
216   return new Set(
217     (daten.ortRuns || [])
218       .map(r => r.ort)
219       .filter(ort => !istKapitel1Unterdrueckt(ort, daten))
220   );
221 }
222
223 // Anteil 0..1 der Scrollstrecke (9300vh, .scroll-track in index.html).
224 // ACHTUNG bei geänderter Streckenlänge alle Werte umrechnen, sonst
225 // verschieben sich die Akte gegeneinander.
226 const SCROLL_MEILENSTEINE = {
227   heroFadeStart: 0.011435, heroFadeEnd: 0.034304,
228   // 960vh davor für den zwölfteiligen Intro-Crawl (.begleittext in
229   // index.html), 80vh je Text, danach 874vh Legendenaufbau.
230   // Zoomdauer unverändert 440vh.
231   zoomStart: 0.231809, zoomEnd: 0.277547,
232   // Crossfade Startkarte -> helle Überblickskarte, 80vh.
233   kartenwechselStart: 0.132661, kartenwechselEnd: 0.140977,
234   // Demo-Kreisgrafik: wächst über den Kreisgrößen-Schritt heran und schrumpft
235   // ab zoomStart auf den Icon-Platz oben rechts. Davor steht nur der
236   // Mittelpunkt mit seiner Ortsbeschriftung da (PDF-Seite 1); der Schleier
237   // blendet in dieser Zeit ein, also von kartenwechselEnd bis demoStart.
238   demoStart: 0.149293, demoVoll: 0.157609,
239   // Legendenaufbau: neun Stufen à 80vh (PDF-Seiten 1-9), gesteuert von den neun
240   // data-demo-gruppe-Texten in index.html. Ab legendeSchleierAus blendet der
241   // Schleier über 80vh aus, erst danach beginnt der Zoom.
242   legendeSchleierAus: 0.220301, legendeEnde: 0.228617,
243   // Spine blendet gleichzeitig mit dem Zoom-Beginn ein.
244   spineFadeStart: 0.109356, spineFadeEnd: 0.155094,
245   // 200vh Lesezeit davor für den Kapitel-Einstiegstext.
246   routeStart: 0.298337, routeEnd: 0.412682,
247   // Kapitel-1-Ende: ab routeEnd der Projekttext-Einblender (140vh), ab
248   // kapitelEndeStart die Kartenansicht mit Hinweis und den beiden Buttons
249   // (100vh bis zur Klemme).
250   kapitelEndeStart: 0.427235,
251   // Übersichtsrouten 02-18. 2933vh breit, damit auch Kapitel 8 auf
252   // ~7.3vh/Annotation kommt wie Kapitel 1.
253
254   // ACHTUNG uebersichtRoutenStart ist zugleich die Klemme am Ende von
255   // Kapitel 1: weiter scrollt draw() nicht. Einen Rauszoom-Akt gibt es nicht
256   // mehr, in den Übersichtsakt führt nur noch ein Klick auf "Übersicht" oder
257   // "Alle".
258   uebersichtRoutenStart: 0.437630, uebersichtRoutenEnd: 0.742516,
259   // Schlussakt Ortsveränderung (2000vh): Kreise der sieben VERGLEICHS_KNOTEN
260   // wachsen mit jedem Kapitel. kreisVergleichFadeEnd liest niemand.
261   kreisVergleichStart: 0.742516, kreisVergleichFadeEnd: 0.759252,
262   kreisVergleichEnd: 0.950416,
```

```

263 // Die Startkarte kommt zurück und zoomt auf die Gesamtkarte raus.
264 startkarteStart: 0.950416,
265 };
266
267 // -----
268 // Datenbereinigung (läuft einmal in setup(), bevor gezeichnet wird)
269 // -----
270
271 function bereinigeStationenDaten(rohdaten) {
272     const arrayFuer = (wert) => Array.isArray(wert) ? wert : Object.values(wert || {});
273
274     rohdaten.route = arrayFuer(rohdaten.route);
275     rohdaten.gedanken = arrayFuer(rohdaten.gedanken);
276     rohdaten.markierungen = arrayFuer(rohdaten.markierungen);
277     rohdaten.routenPunkte = arrayFuer(rohdaten.routenPunkte);
278     rohdaten.annotationen = arrayFuer(rohdaten.annotationen).filter(a => !a.deaktiviert);
279     rohdaten.ortRuns = arrayFuer(rohdaten.ortRuns);
280
281     return rohdaten;
282 }
283
284 function bereinigeFotoMarker(rohdaten) {
285     return Array.isArray(rohdaten) ? rohdaten : Object.values(rohdaten || {});
286 }
287
288 // Filtriert Kapitel ohne Routenpunkte. Schwelle bei EINEM Punkt, nicht zwei –
289 // am selben Objekt hängen auch Kapitelpunkt, Scheibe und Einstiegstext.
290 function bereinigeUebersichtsrouten(rohdaten) {
291     let bereinigt = {};
292     Object.entries(rohdaten || {}).forEach(([kapitel, punkte]) => {
293         if (Array.isArray(punkte) && punkte.length > 0) bereinigt[kapitel] = punkte;
294     });
295     return bereinigt;
296 }
297
298 // -----
299 // Kreis-Radius / Kreispunkte
300 // -----
301
302 // Flächenproportional (sqrt), Standard bei proportional symbol maps: die
303 // Fläche wächst linear mit n. maxRadius deckelt, der Schlussakt gibt Infinity.
304 function kreisRadius(n, maxRadius = 100) {
305     const BASIS = 6, K = 15.5;
306     return n > 0 ? Math.min(maxRadius, BASIS + K * Math.sqrt(n)) : 0;
307 }
308
309 // Aussenradius des ganzen Kreisdiagramms, an dem die F-Wert-Punkte ansetzen.
310
311 // ACHTUNG umgekehrte Parameterreihenfolge: zeichneKreiseFuerRun() nimmt
312 // (..., radiusSkala, maxRadius), hier steht (..., maxRadius, radiusSkala).
313 function groessterKreisRadius(bandCounts, maxRadius = 100, radiusSkala = 1) {
314     let groesster = 0;
315     KREIS_KATEGORIEN.forEach(kat => {
316         let b = bandCounts[kat.key] || {};
317         let n = (b.neg || 0) + (b.pos || 0) + (b.neutral || 0) + (b.unrated || 0);
318         groesster = Math.max(groesster, kreisRadius(n, maxRadius) * radiusSkala);
319     });
320     return groesster;
321 }
322
323 // Ein Ort, der im Text nur vorab erwähnt wird, bekommt noch keinen Kreis –
324 // den zeichnet die Station an seinem echten Halt.
325 function istVorzeitigeErwaehnung(r, daten = stationenData) {
326     let halteort = (daten.halteorte || []).find(h => h.name === r.ort);
327     return !!halteort && halteort.revealIndex !== r.revealIndex;
328 }

```

```

329
330 // Bekommt dieser ortRun beim aktuellen Scrollstand einen Kartenkreis?
331 // punktIndex steuert das Erscheinen, annIndex den Wohnung-Split.
332 function ortRunSichtbar(r, punktIndex, annIndex, daten = stationenData) {
333   if (punktIndex < r.revealIndex) return false;
334   if (istVorzeitigeErwaehnung(r, daten)) return false;
335   if (istKapitel1Unterdrueckt(r.ort, daten)) return false;
336   // Zeitabhängig, deshalb nicht in istKapitel1Unterdrueckt.
337   if (daten === stationenData && r.ort === RUE_NOTRE_DAME_DE_LORETTE_ORT
338     && annIndex < wohnungSplitAi(daten)) return false;
339   return true;
340 }
341
342 // ACHTUNG muss mit valenz_bucket() in baue-kapitel-stationen.py
343 // übereinstimmen: vorberechnete bandCounts kommen von dort, live gezählte
344 // entstehen hier.
345 function valenzBucket(v) {
346   if (v === 1) return 'pos';
347   if (v === -1) return 'neg';
348   if (v === 0) return 'neutral';
349   return 'unrated';
350 }
351
352 // Rohe Trefferliste statt Zählung. Die F-Wert-Punkte brauchen je Annotation
353 // Valenz und fWertType, was eine Aggregation nicht mehr hergibt.
354 function sammleAnnotationenNachOrtBasis(filter, annIndex, daten = stationenData) {
355   let ortBasisWerte = filter instanceof Set ? filter : new Set([filter]);
356   return daten.annotations.filter((a, ai) => {
357     if (ai > annIndex) return false;
358     let treffer = typeof filter === 'function' ? filter(a, ai)
359       : typeof filter === 'number' ? a.id === filter
360       : ortBasisWerte.has(a.ortBasis || a.ort || '');
361     return treffer && a.category;
362   });
363 }
364
365 // Zählt eine bereits gefilterte Trefferliste zusammen. Getrennt vom Sammeln,
366 // damit Aufrufer mit beidem nur einmal über daten.annotations laufen.
367 function zaehleBandCounts(annotationen) {
368   let ergebnis = {
369     gold_dunkel: { unrated: 0, neg: 0, pos: 0, neutral: 0 },
370     gold_mittel: { unrated: 0, neg: 0, pos: 0, neutral: 0 },
371     gold_hell: { unrated: 0, neg: 0, pos: 0, neutral: 0 },
372   };
373   annotationen.forEach(a => {
374     ergebnis[a.category][valenzBucket(a.valenz)]++;
375   });
376   return ergebnis;
377 }
378
379 // Sammeln und Zählen in einem Aufruf, für Stellen ohne Bedarf an der Liste.
380 function zaehleAnnotationenLiveNachOrtBasis(filter, annIndex, daten = stationenData) {
381   return zaehleBandCounts(sammleAnnotationenNachOrtBasis(filter, annIndex, daten));
382 }
383
384 // -----
385 // Spine-Daten
386 // -----
387
388 // Ein Eintrag je zusammenhängendem Besuch. Läuft über daten.annotations,
389 // nicht über daten.ortRuns: nur dort ist eine Rückkehr erkennbar.
390 function baueSpineDaten(daten, hauptorte) {
391   let eintraege = [];
392   let indexNachOrt = new Map(); // ortBasis-Name -> Index in eintraege
393   let laufenderOrt = null; // welcher zusammenhängende Besuch gerade läuft
394

```

```
395 (daten.annotationen || []).forEach((a, ai) => {
396   let ort = a.ortBasis || a.ort || '';
397   if (!hauptorte.has(ort)) { laufenderOrt = null; return; }
398
399   if (laufenderOrt === ort) return; // derselbe Besuch läuft weiter, kein neuer Eintrag
400   laufenderOrt = ort;
401
402   if (indexNachOrt.has(ort)) {
403     eintraege.push({ typ: 'rueckkehr', text: ort, rv: ai, zielIndex: indexNachOrt.get(ort) });
404   } else {
405     indexNachOrt.set(ort, eintraege.length);
406     eintraege.push({ typ: 'location', text: ort, rv: ai, ortBasis: ort });
407   }
408 });
409
410 return eintraege;
411 }
412
413
414 // --- Export -----
415 // 32 Namen, die grösste Schnittstelle im Projekt. Leser: docs/architektur.md.
416
417 // Farben, Kategorien, Punktgrößen
418 window.CATEGORY_COLORS = CATEGORY_COLORS;
419 window.CATEGORY_LABELS = CATEGORY_LABELS;
420 window.KREIS_KATEGORIEN = KREIS_KATEGORIEN;
421 window.ROUTE_COLOR = ROUTE_COLOR;
422 window.ROUTE_COLOR_RGB = ROUTE_COLOR_RGB;
423 window.FWERT_COLOR = FWERT_COLOR;
424 window.FWERT_COLOR_RGB = FWERT_COLOR_RGB;
425 window.FWERT_PUNKTGROESSE = FWERT_PUNKTGROESSE;
426 window.FWERT_PUNKT_DURCHMESSER = FWERT_PUNKT_DURCHMESSER;
427 window.FWERT_LABELS = FWERT_LABELS;
428 window.WAHRNEHMUNG_LABELS = WAHRNEHMUNG_LABELS;
429 window.LEGENDE_BLOCK_TITEL = LEGENDE_BLOCK_TITEL;
430 window.LEGENDE_KREISGROESSE = LEGENDE_KREISGROESSE;
431 window.LEGENDE_VALENZ = LEGENDE_VALENZ;
432 window.LEGENDE_ORTSBESCHRIFTUNG = LEGENDE_ORTSBESCHRIFTUNG;
433 window.LEGENDE_TITEL = LEGENDE_TITEL;
434 window.LEGENDE_UNTERTITEL = LEGENDE_UNTERTITEL;
435 window.FOTO_MARKER_FARBE_RGB = FOTO_MARKER_FARBE_RGB;
436 window.FOTO_MARKER_KERN_FARBE_RGB = FOTO_MARKER_KERN_FARBE_RGB;
437 window.SCHRIFT_SANS = SCHRIFT_SANS;
438 window.SCHRIFT_SERIF = SCHRIFT_SERIF;
439
440 // Stammdaten: welche Kapitel, welche Scroll-Marken, welcher Sammelpunkt
441 window.KAPITEL_MIT_SPINE_PANEL = KAPITEL_MIT_SPINE_PANEL;
442 window.SCROLL_MEILENSTEINE = SCROLL_MEILENSTEINE;
443 window.WOHNUNG_SAMMELPUNKT_ANKER = WOHNUNG_SAMMELPUNKT_ANKER;
444
445 // Eingangsdaten bereinigen (nur preload/setup in sketch.js)
446 window.bereinigeStationenDaten = bereinigeStationenDaten;
447 window.bereinigeFotoMarker = bereinigeFotoMarker;
448 window.bereinigeUebersichtsrouten = bereinigeUebersichtsrouten;
449
450 // Annotationen sammeln und zählen (die heissen Pfade, siehe draw())
451 window.sammleAnnotationenNachOrtBasis = sammleAnnotationenNachOrtBasis;
452 window.zaehleBandCounts = zaehleBandCounts;
453 window.zaehleAnnotationenLiveNachOrtBasis = zaehleAnnotationenLiveNachOrtBasis;
454
455 // Kreisgeometrie und Farbumrechnung
456 window.hexZuRgb = hexZuRgb;
457 window.rgbZuHex = rgbZuHex;
458 window.kreisRadius = kreisRadius;
459 window.groessterKreisRadius = groessterKreisRadius;
460
```

```
461 // Orte, Sichtbarkeit, Spine-Aufbau
462 window.wohnungFilterFuerOrt = wohnungFilterFuerOrt;
463 window.ortRunSichtbar = ortRunSichtbar;
464 window.ortRunsFuerSpine = ortRunsFuerSpine;
465 window.baueSpineDaten = baueSpineDaten;
466
467 }()); // Ende der Modulkapselung, siehe Kommentar oben
```